

Dự báo giá cổ phiếu các ngân hàng thương mại Việt Nam sử dụng Machine Learning và Deep Learning

1. Nguyễn Ngọc Lợi
MSSV: 23520871

2. Nguyễn Tri An
MSSV: 23520021

3. Thiều Đình Nam Tài
MSSV: 23521378

4. Nguyễn Lan Hương
MSSV: 23520588

Giảng viên hướng dẫn: ThS. Dương Phi Long

*Trường Đại học Công nghệ Thông tin
Đại học Quốc gia TP.HCM, Việt Nam*

Tóm tắt nội dung—Trong bối cảnh thị trường chứng khoán Việt Nam ngày càng phát triển, nhóm cổ phiếu ngân hàng đóng vai trò quan trọng do có vốn hóa lớn, thanh khoản cao và phản ánh tương đối rõ nét xu hướng chung của thị trường. Việc dự báo chính xác giá cổ phiếu ngân hàng không chỉ hỗ trợ nhà đầu tư trong việc ra quyết định mà còn góp phần nâng cao hiệu quả quản trị rủi ro. Đề tài tập trung nghiên cứu và so sánh các phương pháp dự báo chuỗi thời gian từ mô hình thống kê truyền thống đến các mô hình học máy và học sâu hiện đại, bao gồm ARIMA, LSTM, GRU, iTransformer và PatchTST. Trên cơ sở dữ liệu giá cổ phiếu của ba ngân hàng thương mại lớn tại Việt Nam (BIDV, Vietcombank và TPBank), nhóm tiến hành huấn luyện mô hình và đánh giá hiệu quả dự báo trong các khoảng thời gian 30, 60 và 90 ngày. Kết quả thực nghiệm cho thấy PatchTST vượt trội hơn các mô hình còn lại với sai số thấp và khả năng dự báo dài hạn tốt.

Index Terms—Time Series Forecasting, Stock Price Prediction, ARIMA, LSTM, GRU, iTransformer, PatchTST, Deep Learning, Transformer.

I. Giới thiệu

A. Bối cảnh và Đặt vấn đề

Ngân hàng là một trong những nhóm ngành trụ cột của thị trường chứng khoán Việt Nam. Giá cổ phiếu ngân hàng thường có mức biến động tương đối ổn định, dữ liệu lịch sử phong phú và phản ánh nhanh các biến động kinh tế vĩ mô. Trong bối cảnh đó, nhu cầu dự báo giá cổ phiếu nhằm hỗ trợ đầu tư và quản lý rủi ro ngày càng trở nên cấp thiết.

Sự phát triển mạnh mẽ của trí tuệ nhân tạo, đặc biệt là các mô hình học máy và học sâu, đã mở ra nhiều hướng tiếp cận mới cho bài toán dự báo chuỗi thời gian. Tuy nhiên, dữ liệu tài chính vốn phức tạp, nhiễu và chịu ảnh hưởng của nhiều yếu tố khiến việc lựa chọn mô hình phù hợp trở thành một thách thức lớn. Do đó, việc nghiên cứu, so sánh và đánh giá hiệu quả của các mô hình dự báo khác nhau là cần thiết và có ý nghĩa thực tiễn cao.

Việc dự báo chính xác giá cổ phiếu ngân hàng có ý nghĩa quan trọng đối với nhiều bên liên quan. Đối với nhà đầu tư cá nhân và tổ chức, dự báo chính xác giúp đưa ra quyết định mua/bán hợp lý, tối ưu hóa lợi nhuận và quản lý rủi ro. Đối với các quỹ đầu tư và công ty quản lý tài sản, dự báo giá cổ

phiếu là cơ sở để xây dựng chiến lược đầu tư và phân bổ danh mục. Đối với các nhà hoạch định chính sách, việc hiểu được xu hướng giá cổ phiếu ngân hàng giúp đánh giá sức khỏe của hệ thống tài chính và đưa ra các chính sách phù hợp.

Tuy nhiên, giá cổ phiếu chịu ảnh hưởng của nhiều yếu tố phức tạp và đa chiều. Các yếu tố vĩ mô như lãi suất, tỷ giá, lạm phát, và tăng trưởng GDP có tác động trực tiếp đến hoạt động kinh doanh của ngân hàng và do đó ảnh hưởng đến giá cổ phiếu. Các yếu tố vi mô như kết quả kinh doanh, tỷ lệ nợ xấu, và chiến lược phát triển của từng ngân hàng cũng đóng vai trò quan trọng. Ngoài ra, tâm lý thị trường, tin tức, và các sự kiện bất ngờ (như đại dịch COVID-19) có thể gây ra những biến động lớn và khó dự đoán. Tính chất phi tuyến, nhiễu, và phụ thuộc thời gian của dữ liệu giá cổ phiếu khiến việc dự báo trở thành một thách thức lớn.

Các phương pháp dự báo truyền thống dựa trên phân tích kỹ thuật và mô hình thống kê như ARIMA đã được sử dụng rộng rãi trong nhiều thập kỷ. ARIMA có ưu điểm là đơn giản, dễ hiểu, và không yêu cầu nhiều dữ liệu. Tuy nhiên, mô hình này thường gặp hạn chế trong việc nắm bắt các mẫu phi tuyến phức tạp và các phụ thuộc dài hạn trong dữ liệu. ARIMA giả định tính dừng của chuỗi thời gian và có xu hướng dự báo hội tụ về giá trị trung bình trong dài hạn, không phù hợp với tính chất biến động cao của giá cổ phiếu.

Sự phát triển của học máy và học sâu trong những năm gần đây đã mở ra những hướng tiếp cận mới cho bài toán dự báo chuỗi thời gian. Các mô hình như LSTM (Long Short-Term Memory) và GRU (Gated Recurrent Unit) được thiết kế đặc biệt để xử lý dữ liệu tuần tự và có khả năng học được các phụ thuộc thời gian phức tạp thông qua cơ chế gating. Tuy nhiên, các mô hình RNN này vẫn có hạn chế về khả năng học các phụ thuộc rất dài hạn do vấn đề vanishing gradient và độ phức tạp tính toán tăng theo độ dài chuỗi.

Gần đây, các kiến trúc Transformer được thiết kế đặc biệt cho chuỗi thời gian như iTransformer và PatchTST đã cho thấy tiềm năng vượt trội trong việc dự báo dài hạn. iTransformer đảo ngược cách áp dụng Attention từ chiều thời gian sang chiều biến, giúp học được mối quan hệ giữa các biến một cách hiệu quả. PatchTST áp dụng kỹ thuật Patching từ Vision

Transformer, chia chuỗi thời gian thành các mảng con để giảm độ phức tạp tính toán và học được cả thông tin cục bộ lẫn phụ thuộc dài hạn. Các mô hình này đã đạt được kết quả ấn tượng trên nhiều bộ dữ liệu chuỗi thời gian, nhưng chưa được đánh giá kỹ lưỡng trên dữ liệu cổ phiếu ngân hàng Việt Nam.

B. Mục tiêu đồ án

Mục tiêu tổng quát: Nghiên cứu và so sánh hiệu quả của các mô hình học máy/học sâu trong bài toán dự báo giá cổ phiếu ngân hàng Việt Nam.

Mục tiêu cụ thể:

- 1) **Khảo sát các phương pháp:** Tìm hiểu các mô hình từ truyền thống (ARIMA) đến hiện đại (Transformer), đặc biệt tập trung vào iTransformer và PatchTST
- 2) **Xây dựng mô hình dự báo:** Triển khai năm mô hình: ARIMA, LSTM, GRU, iTransformer, và PatchTST trên dữ liệu giá cổ phiếu ngân hàng
- 3) **Đánh giá và so sánh:** So sánh hiệu suất các mô hình trên nhiều cổ phiếu và tỷ lệ train/test khác nhau (70:30, 80:20, 90:10)
- 4) **Dự báo tương lai:** Dự đoán giá cổ phiếu trong 30, 60, và 90 ngày tiếp theo để đánh giá khả năng dự báo dài hạn
- 5) **Đề xuất mô hình tối ưu:** Xác định mô hình phù hợp nhất cho bài toán dự báo giá cổ phiếu ngân hàng Việt Nam

C. Câu hỏi nghiên cứu

Nghiên cứu này nhằm trả lời các câu hỏi sau:

- 1) Mô hình nào cho kết quả dự báo tốt nhất trên dữ liệu cổ phiếu ngân hàng Việt Nam?
- 2) Các mô hình Transformer (iTransformer, PatchTST) có vượt trội so với các mô hình truyền thống (ARIMA) và Deep Learning cổ điển (LSTM, GRU)?
- 3) Tỷ lệ chia train/test ảnh hưởng như thế nào đến hiệu suất dự báo?
- 4) Khả năng dự báo dài hạn (30-90 ngày) của các mô hình như thế nào?

D. Phạm vi và Đối tượng nghiên cứu

Đối tượng nghiên cứu:

- **Dữ liệu:** Giá đóng cửa hàng ngày của ba cổ phiếu ngân hàng
- **Cổ phiếu:** BIDV (BID), Vietcombank (VCB), và TPBank (TPB)
- **Nguồn dữ liệu:** Investing.com
- **Thời gian:** 01/01/2019 - 27/11/2025 (khoảng 7 năm, 1,725 điểm dữ liệu mỗi cổ phiếu)

Phạm vi nghiên cứu:

- **Loại dự báo:** Univariate (chỉ sử dụng giá đóng cửa)
- **forecast horizon:** 1 ngày (training), 30/60/90 ngày (testing)
- **Mô hình:** Năm mô hình (ARIMA, LSTM, GRU, iTransformer, PatchTST)
- **Tỷ lệ train/test:** 70:30, 80:20, 90:10

Giới hạn nghiên cứu:

- Chỉ sử dụng dữ liệu giá (không bao gồm tin tức, sentiment, chỉ số vĩ mô)
- Không xem xét chi phí giao dịch trong đánh giá
- Tập trung vào nhóm ngân hàng, chưa mở rộng sang các ngành khác

E. Ý nghĩa nghiên cứu

Ý nghĩa khoa học:

- Đánh giá thực nghiệm các mô hình Transformer thế hệ mới (iTransformer, PatchTST) trên dữ liệu chứng khoán Việt Nam
- Cung cấp benchmark cho các nghiên cứu tương lai về dự báo giá cổ phiếu
- Phân tích điểm mạnh/yếu của từng phương pháp trong bối cảnh thị trường Việt Nam

Ý nghĩa thực tiễn:

- **Nhà đầu tư cá nhân:** Công cụ hỗ trợ ra quyết định đầu tư dựa trên dự báo khoa học
- **Quỹ đầu tư:** Tham khảo cho xây dựng chiến lược trading và quản lý danh mục
- **Sinh viên/Nghiên cứu sinh:** Tài liệu tham khảo về Time Series Forecasting và ứng dụng trong tài chính
- **Công ty Fintech:** Cơ sở phát triển sản phẩm dự báo tài chính và robo-advisor

II. Cơ sở lý thuyết

A. Mô hình ARIMA

ARIMA (AutoRegressive Integrated Moving Average) là mô hình thống kê truyền thống dựa trên ba thành phần: tự hồi quy (AR), sai phân (I) và trung bình trượt (MA) [1]. Mô hình kết hợp ba thành phần chính:

- **AR (AutoRegressive):** Sử dụng các giá trị quá khứ của chuỗi để dự báo giá trị hiện tại. Phần này mô hình hóa mối quan hệ giữa giá trị hiện tại và các giá trị trong quá khứ.
- **I (Integrated):** Áp dụng sai phân để làm cho chuỗi thời gian trở nên dừng (stationary). Điều này giúp loại bỏ xu hướng và tính mùa vụ trong dữ liệu.
- **MA (Moving Average):** Sử dụng các sai số trong quá khứ để cải thiện dự báo. Phần này giúp mô hình học được các pattern trong nhiễu của chuỗi thời gian.

Mô hình ARIMA được ký hiệu là $ARIMA(p, d, q)$, trong đó:

- **p (AutoRegressive order):** Bậc của phần tự hồi quy, cho biết số lượng giá trị trong quá khứ được sử dụng để dự báo giá trị hiện tại
- **d (Differencing order):** Bậc sai phân, số lần lấy sai phân của chuỗi thời gian để đạt được tính dừng
- **q (Moving Average order):** Bậc của phần trung bình động, số lượng nhiễu trắng (white noise) trong quá khứ được sử dụng

Quy trình thực hiện ARIMA:

- 1) **Kiểm tra tính dừng:** Sử dụng kiểm định Augmented Dickey-Fuller (ADF) để xác định xem chuỗi thời gian

có tính dừng hay không. Nếu chuỗi không dừng, cần áp dụng các phép biến đổi.

- 2) **Biến đổi chuỗi:** Áp dụng log transformation và differencing để làm cho chuỗi trở nên dừng. Bậc sai phân d được xác định bằng số lần cần lấy sai phân để đạt được tính dừng.
- 3) **Xác định tham số:** Trong nghiên cứu này, tham số p và q được lựa chọn tự động bằng auto-ARIMA, với bậc sai phân $d = 1$. Auto-ARIMA thực hiện grid search trên không gian tham số và chọn mô hình có AIC (Akaike Information Criterion) thấp nhất.
- 4) **Huấn luyện mô hình:** Sau khi xác định được các tham số tối ưu, mô hình được huấn luyện trên tập dữ liệu train và sử dụng để dự báo.

Kết quả Auto-ARIMA:

- BIDV: ARIMA(0,1,0) - Mô hình random walk với sai phân bậc 1
- VCB: ARIMA(0,1,0) - Tương tự BIDV
- TPBank: ARIMA(2,1,0) - Mô hình tự hồi quy bậc 2 với sai phân bậc 1

B. Mô hình LSTM

LSTM (Long Short-Term Memory) là một biến thể của mạng neural hồi quy được thiết kế để giải quyết vấn đề biến mất gradient trong RNN truyền thống [2]. Kiến trúc LSTM sử dụng các cổng (gates) để kiểm soát luồng thông tin:

- **Forget Gate:** Quyết định thông tin nào cần loại bỏ
- **Input Gate:** Quyết định thông tin mới nào cần lưu trữ
- **Output Gate:** Quyết định thông tin nào từ cell state được đưa ra output

Công thức toán học của LSTM cell:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{Forget gate}) \quad (1)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{Input gate}) \quad (2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (3)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (\text{Cell state}) \quad (4)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{Output gate}) \quad (5)$$

$$h_t = o_t * \tanh(C_t) \quad (6)$$

Trong đó σ là hàm sigmoid, $*$ là phép nhân element-wise, và W, b là các ma trận trọng số và bias cần học.

Kiến trúc LSTM trong nghiên cứu:

```
model = Sequential()
model.add(LSTM(50, return_sequences=True,
               input_shape=(100, 1)))
model.add(LSTM(50, return_sequences=True))
model.add(LSTM(50))
model.add(Dense(1))
```

Listing 1. Kiến trúc LSTM

Trong nghiên cứu này, nhóm sử dụng kiến trúc LSTM 3 lớp với 50 units mỗi lớp, input shape (100, 1) tương ứng với 100 ngày lịch sử. Kiến trúc nhiều lớp giúp mô hình học được các biểu diễn phân cấp từ mức thấp (pattern ngắn hạn) đến mức cao (xu hướng dài hạn). Cả hai mô hình LSTM và GRU đều được huấn luyện bằng optimizer Adam và hàm mất mát MSE (Mean Squared Error).

C. Mô hình GRU

GRU (Gated Recurrent Unit) là một biến thể đơn giản hóa của LSTM với ít tham số hơn [3]. GRU chỉ sử dụng hai cổng:

- **Reset Gate:** Quyết định lượng thông tin quá khứ cần bỏ qua
- **Update Gate:** Quyết định lượng thông tin mới cần cập nhật

Công thức toán học của GRU cell:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (\text{Reset gate}) \quad (7)$$

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (\text{Update gate}) \quad (8)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t] + b) \quad (9)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \quad (10)$$

GRU có kiến trúc đơn giản hơn LSTM với chỉ một lớp 64 units. Kiến trúc GRU trong nghiên cứu:

```
model = Sequential()
model.add(GRU(64, input_shape=(100, 1)))
model.add(Dense(1))
```

Listing 2. Kiến trúc GRU

GRU có ít tham số hơn LSTM (không có cell state riêng biệt) nhưng vẫn giữ được khả năng học các phụ thuộc dài hạn thông qua cơ chế update gate. Mặc dù đơn giản hơn LSTM, GRU thường đạt hiệu suất tương đương trong nhiều bài toán và có tốc độ huấn luyện nhanh hơn do ít phép tính hơn. Nhóm sử dụng GRU với 1 lớp và 64 units để so sánh với LSTM và đánh giá sự đánh đổi giữa độ phức tạp và hiệu suất. Cả hai mô hình đều được huấn luyện với 100 epochs và batch size 64.

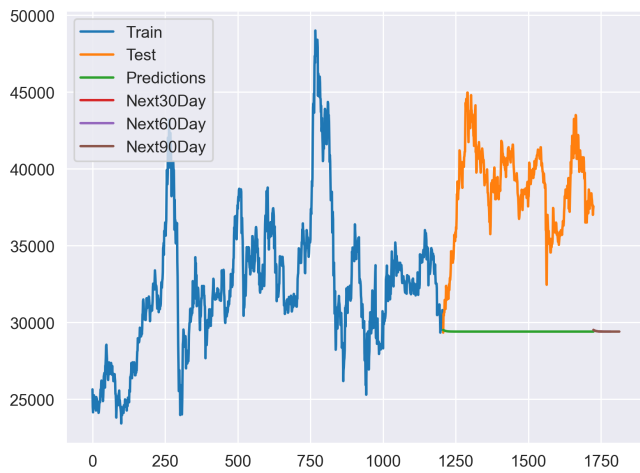
D. Kết quả các mô hình ARIMA, LSTM và GRU

Các mô hình ARIMA, LSTM và GRU được đánh giá bằng các chỉ số MAE, MAPE và RMSE trên ba cổ phiếu BIDV, VCB và TPBank. Kết quả được trình bày trong các bảng dưới đây, mỗi bảng hiển thị kết quả của một ngân hàng với ba mô hình.

- 1) **Kết quả trên cổ phiếu BIDV:** Các hình 10, 11, và 12 minh họa kết quả dự báo của ba mô hình ARIMA, LSTM và GRU trên cổ phiếu BIDV.
- 2) **Kết quả trên cổ phiếu VCB:** Các hình 4, 5, và 6 minh họa kết quả dự báo của ba mô hình ARIMA, LSTM và GRU trên cổ phiếu VCB.
- 3) **Kết quả trên cổ phiếu TPBank:** Các hình 7, 8, và 9 minh họa kết quả dự báo của ba mô hình ARIMA, LSTM và GRU trên cổ phiếu TPBank.

E. Mô hình iTransformer

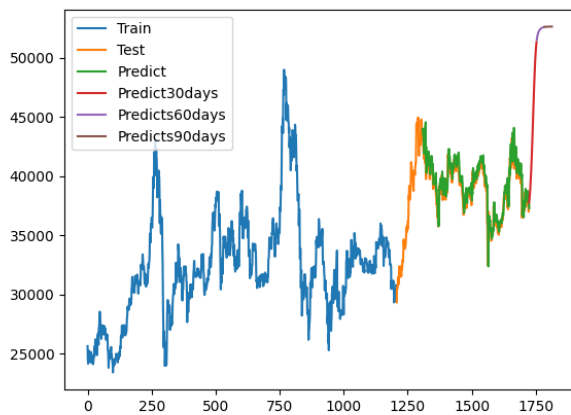
iTransformer (Inverted Transformer) là một biến thể của kiến trúc Transformer truyền thống, được đề xuất nhằm khắc phục các hạn chế khi áp dụng Transformer trực tiếp cho bài toán dự báo chuỗi thời gian, đặc biệt là chuỗi thời gian đa biến (multivariate time series) [4].



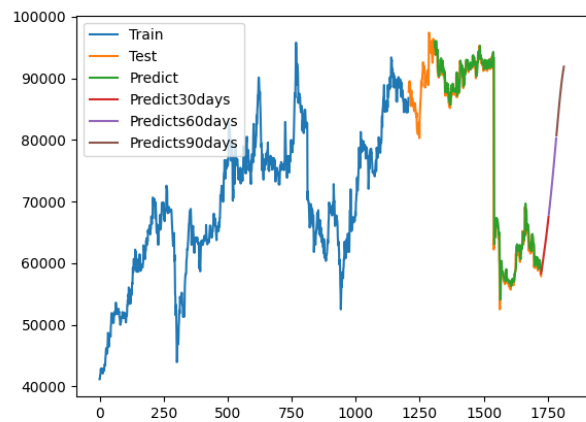
Hình 1. ARIMA - BIDV



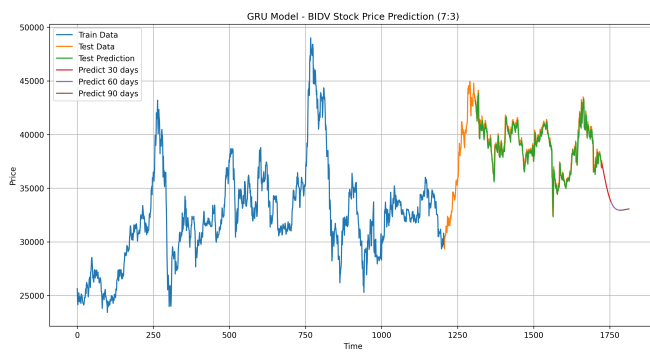
Hình 4. ARIMA - VCB



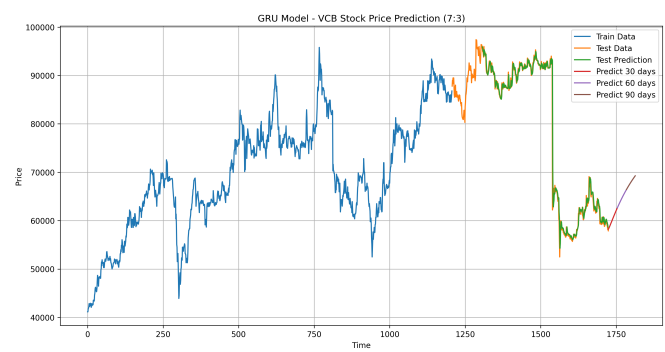
Hình 2. LSTM - BIDV



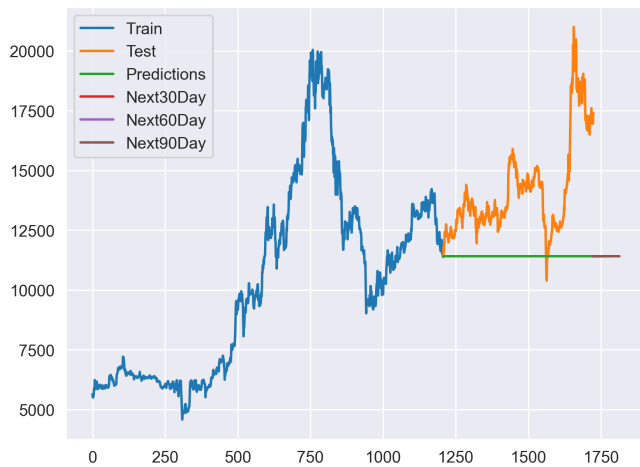
Hình 5. LSTM - VCB



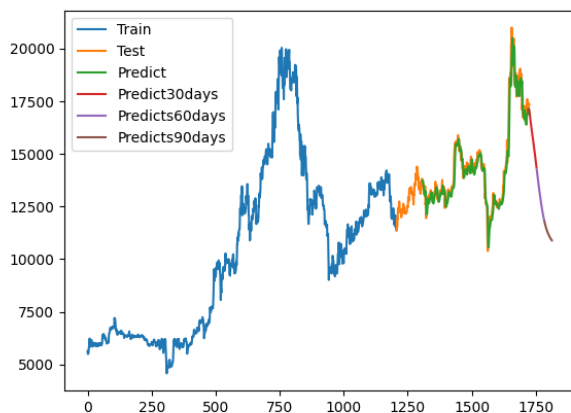
Hình 3. GRU - BIDV



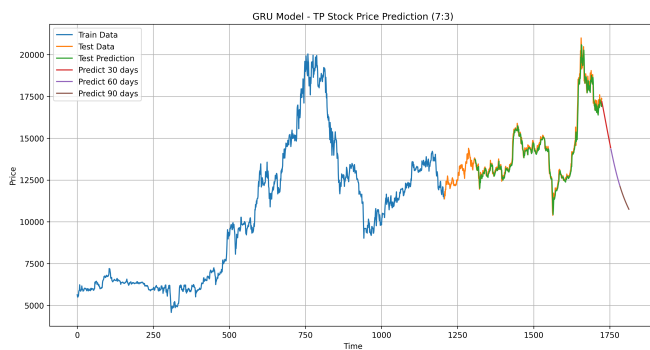
Hình 6. GRU - VCB



Hình 7. ARIMA - TPBank



Hình 8. LSTM - TPBank



Hình 9. GRU - TPBank

1) *Hạn chế của Transformer truyền thống trong dự báo chuỗi thời gian:* Transformer truyền thống được thiết kế cho các bài toán xử lý ngôn ngữ tự nhiên (NLP), trong đó mỗi token đại diện cho một từ trong câu. Khi áp dụng cho chuỗi thời gian, cách tiếp cận phổ biến là coi mỗi thời điểm (time step) là một token, còn các biến (features) được gộp vào cùng một vector. Cách làm này tồn tại nhiều hạn chế:

- **Giả định không phù hợp:** Giả định rằng tất cả các biến tại cùng một thời điểm phản ánh cùng một trạng thái hoặc sự kiện, trong khi thực tế các biến tài chính (giá, khối lượng, chỉ báo kỹ thuật) có bản chất và ý nghĩa khác nhau.
- **Khó khai thác tương quan giữa biến:** Cơ chế Self-Attention được áp dụng theo chiều thời gian (temporal dimension) khiến mô hình khó khai thác mối tương quan giữa các biến.
- **Hiệu suất không cải thiện với chuỗi dài:** Hiệu suất dự báo không cải thiện khi tăng độ dài chuỗi đầu vào (lookback window), thậm chí có thể suy giảm, trái ngược với các mô hình tuyến tính vốn hưởng lợi từ dữ liệu lịch sử dài hơn.
- **Chi phí tính toán cao:** Kiến trúc không được thiết kế chuyên biệt cho chuỗi thời gian, dẫn đến chi phí tính toán cao nhưng hiệu quả không tương xứng.

2) *Ý tưởng cốt lõi của iTransformer:* iTransformer giải quyết các hạn chế trên bằng cách đảo ngược (invert) cách áp dụng các thành phần của Transformer. Thay vì token hóa theo thời gian, iTransformer token hóa theo biến. Ba ý tưởng chính bao gồm:

(1) **Variate Tokenization (Token hóa theo biến):** Trong iTransformer, mỗi biến (ví dụ: giá đóng cửa, khối lượng giao dịch hoặc toàn bộ chuỗi giá của một cổ phiếu) được xem như một token. Như vậy:

- Transformer truyền thống: 1 thời điểm = 1 token
- iTransformer: 1 biến = 1 token

Cách tiếp cận này giúp mô hình học tốt hơn mối quan hệ giữa các biến trong toàn bộ chuỗi thời gian.

(2) **Inverted Attention (Attention đảo ngược):** Cơ chế Self-Attention không còn áp dụng trên chiều thời gian mà được áp dụng trên chiều biến. Điều này cho phép mô hình học được câu hỏi quan trọng: biến nào ảnh hưởng đến biến nào, thay vì chỉ học sự phụ thuộc theo thời gian.

(3) **Feed-Forward Network đảo ngược:** Mạng Feed-Forward (FFN) được áp dụng cho biểu diễn chuỗi thời gian của từng biến, giúp mô hình học các mẫu hình phi tuyến phức tạp theo thời gian, bao gồm xu hướng, chu kỳ và biến động bất thường.

Kiến trúc này đặc biệt hiệu quả cho dữ liệu đa biến vì nó học được mối quan hệ giữa các biến một cách trực tiếp thông qua Attention mechanism. Độ phức tạp tính toán của Attention là $O(V^2)$ với V là số biến, thay vì $O(L^2)$ với L là độ dài chuỗi như Transformer truyền thống, giúp giảm đáng kể chi phí tính toán khi chuỗi thời gian dài.

Quy trình xử lý của iTransformer:

- 1) **Transpose:** Chuyển đổi input từ (Batch, Length, Variables) sang (Batch, Variables, Length)
- 2) **Variate Embedding:** Mỗi biến được embed thành một token D chiều
- 3) **Multi-Head Self-Attention:** Áp dụng trên chiều Variables để học quan hệ giữa các biến
- 4) **Feed-Forward Network:** Áp dụng trên chiều Length để học pattern thời gian
- 5) **Projection Head:** Dự báo giá trị tương lai

3) Kiến trúc và cấu hình mô hình iTransformer:

iTransformer vẫn giữ các thành phần cơ bản của Transformer như Multi-Head Attention, Residual Connection và Layer Normalization, nhưng thay đổi thứ tự và chiều áp dụng. Các thông số chính được sử dụng trong nghiên cứu bao gồm:

- Số khối Transformer: 4
- Số head attention: 4
- Kích thước head: 256
- Feed-forward dimension: 4
- Learning rate: 0.0001
- Epochs: 100
- Batch size: 64
- Dropout: 0.25

4) **Đánh giá mô hình iTransformer:** Trong thực nghiệm, iTransformer cho thấy khả năng mô hình hóa mối tương quan giữa các biến tốt hơn Transformer truyền thống. Tuy nhiên, với bài toán dự báo giá cổ phiếu đơn biến (univariate), lợi thế của iTransformer chưa được khai thác tối đa, dẫn đến sai số vẫn còn cao so với PatchTST. Điều này cho thấy iTransformer phù hợp hơn với dữ liệu đa biến nơi có nhiều biến tương quan với nhau.

F. Mô hình PatchTST

PatchTST (Patch Time Series Transformer) là một mô hình học sâu tiên tiến, được thiết kế chuyên biệt cho bài toán dự báo chuỗi thời gian dài hạn, được công bố tại ICLR 2023 [5]. PatchTST xuất phát từ quan điểm coi chuỗi thời gian như một dạng "ngôn ngữ", trong đó mỗi đoạn dữ liệu mang ý nghĩa ngữ nghĩa (semantic meaning) thay vì từng điểm dữ liệu đơn lẻ.

1) **Động cơ và vấn đề của các mô hình trước:** Các mô hình như ARIMA, LSTM, GRU và Transformer truyền thống đều xử lý chuỗi thời gian theo từng điểm riêng lẻ (point-wise). Tuy nhiên, một giá trị đơn lẻ tại một thời điểm thường không đủ để phản ánh xu hướng thị trường. Ngoài ra:

- **ARIMA:** Có xu hướng dự báo phẳng khi dự báo xa, không nắm bắt được các biến động phi tuyến phức tạp.
- **LSTM/GRU:** Gặp vấn đề tích lũy sai số khi dự báo nhiều bước liên tiếp (error accumulation), dẫn đến độ chính xác giảm dần theo thời gian.
- **Transformer truyền thống:** Tốn tài nguyên tính toán và không tận dụng tốt cấu trúc chuỗi thời gian, đặc biệt là thông tin cục bộ (local semantic).

2) **Ý tưởng cốt lõi của PatchTST:** PatchTST được xây dựng dựa trên hai ý tưởng chính:

(1) **Patching (Phân mảnh chuỗi thời gian):** Chuỗi thời gian đầu vào được chia thành các đoạn nhỏ (patches), mỗi

patch gồm nhiều điểm liên tiếp (ví dụ: 10 ngày giao dịch). Mỗi patch đóng vai trò như một token, giúp mô hình nắm bắt được xu hướng cục bộ thay vì chỉ một điểm dữ liệu đơn lẻ. Điều này giảm độ phức tạp tính toán từ $O(L^2)$ xuống $O((L/P)^2)$ với P là độ dài patch.

(2) **Channel Independence (Độc lập kênh):** PatchTST xử lý từng chuỗi biến một cách độc lập nhưng chia sẻ cùng một bộ trọng số. Điều này giúp mô hình phù hợp với cả dữ liệu đơn biến và đa biến, đồng thời giảm nguy cơ overfitting và tăng khả năng tổng quát hóa.

3) **Quy trình hoạt động của PatchTST:** Quy trình của PatchTST bao gồm các bước:

- 1) **Chuẩn hóa:** Chuỗi đầu vào được chuẩn hóa bằng Instance Normalization để giảm sự dịch chuyển phân phối dữ liệu.
- 2) **Patching:** Chia chuỗi L điểm thành $N = \lfloor (L-P)/S \rfloor + 1$ patches, mỗi patch có P điểm. Các patch không chồng lấp (non-overlapping) với patch length = 10 và stride = 10.
- 3) **Patch Embedding:** Các patch được ánh xạ vào không gian vector ẩn thông qua lớp Linear và Position Embedding để mô hình biết thứ tự các patches.
- 4) **Transformer Encoder:** Các vector patch được đưa vào Transformer Encoder để học mối quan hệ giữa các đoạn thời gian thông qua Self-Attention và Feed-Forward Network.
- 5) **Flatten & Linear Head:** Đầu ra được làm phẳng và đưa qua Linear Head để dự báo trực tiếp toàn bộ chuỗi tương lai (direct multi-step forecasting), tránh tích lũy sai số.

Ưu điểm của PatchTST:

- **Giảm độ phức tạp:** Từ $O(L^2)$ xuống $O((L/P)^2)$, giảm đáng kể khi L lớn
- **Local semantic:** Mỗi patch giữ lại thông tin cục bộ có ý nghĩa
- **Long-range dependency:** Attention giữa các patches xa nhau giúp học phụ thuộc dài hạn
- **Channel-independence:** Phù hợp với dữ liệu univariate và giúp tổng quát hóa tốt

4) **Ưu điểm của PatchTST so với các mô hình khác:**

- **Giảm nhiễu và giữ ngữ nghĩa:** Thông qua cơ chế patching, PatchTST giảm nhiễu và giữ được ngữ nghĩa xu hướng thay vì chỉ từng điểm dữ liệu đơn lẻ.
- **Tránh tích lũy sai số:** Nhờ dự báo đa bước trực tiếp (direct multi-step forecasting), PatchTST tránh được vấn đề tích lũy sai số khi dự báo nhiều bước liên tiếp.
- **Hiệu quả dự báo dài hạn:** PatchTST đặc biệt hiệu quả trong dự báo dài hạn (30-90 ngày) nhờ khả năng học được long-range dependency thông qua Attention giữa các patches.
- **Phù hợp với dữ liệu tài chính:** PatchTST phù hợp với dữ liệu tài chính có tính phi tuyến và biến động mạnh nhờ cơ chế Channel-Independence và khả năng học pattern phức tạp.

5) *Cấu hình PatchTST trong nghiên cứu:* Các tham số chính được sử dụng bao gồm:

- Look-back window: 100 ngày
- Patch length: 10
- Stride: 10 (non-overlapping)
- d_model: 64
- Number of heads: 4
- Number of encoder layers: 3
- Feed-forward dimension: 256
- Dropout: 0.1
- Optimizer: Adam (learning rate = 0.001)
- Epochs: 100, Batch size: 32

Với sequence length 100, sẽ có 10 patches sau khi patching.

G. Kết quả các mô hình iTransformer và PatchTST

Các mô hình iTransformer và PatchTST được đánh giá bằng các chỉ số MAE, MAPE và RMSE trên ba cổ phiếu BIDV, VCB và TPBank. Kết quả được trình bày trong bảng I.

PatchTST cho kết quả vượt trội trên cả ba cổ phiếu BIDV, VCB và TPBank, với RMSE và MAPE thấp nhất trong tất cả các mô hình được thử nghiệm. Cụ thể, PatchTST đạt RMSE 1,370 cho BIDV, 2,016 cho VCB, và 601 cho TPBank, giảm đáng kể so với iTransformer và các mô hình khác.

III. Phương pháp Đề xuất

A. Quy trình nghiên cứu

Nghiên cứu được thực hiện theo quy trình năm bước như sau:

- 1) **Thu thập dữ liệu:** Tái dữ liệu giá cổ phiếu từ Investing.com cho ba mã cổ phiếu BIDV, VCB, và TPBank từ năm 2019 đến 2025
- 2) **Tiền xử lý & Phân tích EDA:** Làm sạch dữ liệu, kiểm tra tính dừng bằng ADF test, phân tích xu hướng và biến động, chuẩn hóa dữ liệu
- 3) **Xây dựng mô hình:** Triển khai năm mô hình với các cấu hình đã được tối ưu hóa
- 4) **Huấn luyện & Đánh giá:** Huấn luyện từng mô hình trên tập train và đánh giá trên tập test với các metrics RMSE, MAE, và MAPE
- 5) **So sánh & Kết luận:** So sánh hiệu suất các mô hình, phân tích điểm mạnh/yếu, và đề xuất mô hình tối ưu

B. Cách thực hiện của nhóm

Phương pháp tiếp cận: Nhóm áp dụng phương pháp so sánh hiệu quả của các mô hình dự báo chuỗi thời gian từ truyền thống đến hiện đại, với mục tiêu xác định mô hình phù hợp nhất cho bài toán dự báo giá cổ phiếu ngân hàng Việt Nam.

Các mô hình được sử dụng:

- **Mô hình truyền thống:** ARIMA - mô hình thống kê đã được chứng minh hiệu quả trong nhiều thập kỷ
- **Mô hình Deep Learning cổ điển:** LSTM và GRU - các mô hình RNN phổ biến cho dự báo chuỗi thời gian
- **Mô hình Transformer tiên tiến:** iTransformer và PatchTST - các kiến trúc Transformer được thiết kế đặc biệt cho chuỗi thời gian (tập trung nghiên cứu)

Quy trình thực hiện chi tiết:

- 1) Thu thập dữ liệu từ Investing.com với đầy đủ các trường: Date, Price, Open, High, Low, Volume, Change %
- 2) Tiền xử lý dữ liệu: Xử lý missing values, chuẩn hóa bằng MinMaxScaler cho các mô hình học sâu
- 3) Chia tập train/test theo ba tỷ lệ: 70:30, 80:20, và 90:10 để đánh giá ảnh hưởng của tỷ lệ chia dữ liệu
- 4) Huấn luyện và đánh giá từng mô hình với các hyperparameters đã được cấu hình
- 5) Dự báo 30/60/90 ngày tương lai để đánh giá khả năng dự báo dài hạn
- 6) So sánh và phân tích kết quả để rút ra kết luận và đề xuất

C. Dữ liệu

Nghiên cứu sử dụng dữ liệu giá đóng cửa hàng ngày của ba cổ phiếu ngân hàng từ Investing.com:

- **BIDV (BID):** Ngân hàng TMCP Đầu tư và Phát triển Việt Nam
- **VCB:** Ngân hàng TMCP Ngoại thương Việt Nam (Vietcombank)
- **TPBank (TPB):** Ngân hàng TMCP Tiên Phong

Dữ liệu được thu thập từ ngày 01/01/2019 đến 27/11/2025, tổng cộng khoảng 1,725 điểm dữ liệu cho mỗi cổ phiếu. Dataset bao gồm các cột: Date, Price (giá đóng cửa), Open, High, Low, Volume, và Change %.

D. Tiền xử lý dữ liệu

Quá trình tiền xử lý dữ liệu được thực hiện cẩn thận để đảm bảo chất lượng đầu vào cho các mô hình:

1) Làm sạch dữ liệu:

- Loại bỏ các giá trị thiếu (missing values) nếu có
- Xử lý các giá trị bất thường (outliers) bằng cách sử dụng phương pháp IQR (Interquartile Range) hoặc giới hạn giá trị trong khoảng hợp lý
- Kiểm tra tính nhất quán của dữ liệu theo thời gian
- Chuyển đổi định dạng giá từ chuỗi (có dấu phẩy) sang số thực

2) Phân tích khám phá dữ liệu (EDA):

- Kiểm tra tính dừng của chuỗi thời gian bằng Augmented Dickey-Fuller (ADF) test
- Phân tích xu hướng và tính mùa vụ của dữ liệu
- Vẽ biểu đồ phân phối và biểu đồ thời gian để hiểu đặc tính của dữ liệu

3) Chuẩn hóa dữ liệu:

- Đối với các mô hình học sâu (LSTM, GRU, iTransformer, PatchTST): Sử dụng MinMaxScaler để chuẩn hóa giá về khoảng [0, 1] theo công thức:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Việc chuẩn hóa giúp các mô hình học sâu hội tụ nhanh hơn và tránh vấn đề gradient explosion.

- Đối với ARIMA: Không cần chuẩn hóa vì mô hình làm việc trực tiếp với giá trị gốc, nhưng có thể áp dụng log transformation để làm cho chuỗi dừng hơn.

Bảng 1
Kết quả dự báo iTransformer và PatchTST trên ba cổ phiếu ngân hàng

Model	MAE	BIDV MAPE (%)	RMSE	MAE	VCB MAPE (%)	RMSE	MAE	TPBank MAPE (%)	RMSE
iTransformer	39,015.00	6,494,694.33	39,077.92	78,201.53	15,129,944.16	79,669.31	14,228.80	2,450,336.98	14,430.30
PatchTST	1,249.50	3.20	1,369.84	912.63	1.28	2,016.34	459.74	3.07	600.87

4) Tạo sequences:

- Lookback window = 100 ngày: Mỗi mẫu đầu vào bao gồm 100 ngày lịch sử giá đóng cửa
- Output: Giá đóng cửa của ngày tiếp theo (1-step ahead forecasting)
- Với dữ liệu có 1,725 điểm, sau khi tạo sequences sẽ có khoảng 1,625 mẫu (1,725 - 100)

5) Chia tập dữ liệu:

- Tỷ lệ 70:30 (train:test): 1,207 mẫu train và 518 mẫu test
- Chia theo thứ tự thời gian (không shuffle) để đảm bảo tính thực tế của dự báo
- Tập train: Từ ngày 01/01/2019 đến khoảng giữa năm 2023
- Tập test: Từ giữa năm 2023 đến 27/11/2025

E. Cấu hình mô hình

1) *ARIMA*: Sử dụng Auto-ARIMA để tự động tìm tham số tối ưu:

- Phạm vi tìm kiếm: $p \in [0, 5], q \in [0, 5], d = 1$
- Tiêu chí tối ưu: AIC (Akaike Information Criterion)
- Kết quả: BIDV và VCB đạt ARIMA(0,1,0), TPBank đạt ARIMA(2,1,0)

2) *LSTM*:

- Kiến trúc: 3 lớp LSTM với 50 units mỗi lớp
- Input shape: (100, 1)
- Optimizer: Adam với learning rate mặc định
- Loss function: Mean Squared Error (MSE)
- Epochs: 100 với batch size 64

3) *GRU*:

- Kiến trúc: 1 lớp GRU với 64 units
- Input shape: (100, 1)
- Optimizer: Adam
- Loss function: MSE
- Epochs: 100 với batch size 64

4) *iTransformer*:

- Head size: 256
- Số attention heads: 4
- Số lớp Transformer: 4
- Feed-forward dimension: 4
- Dropout: 0.25
- Learning rate: 1e-4
- Epochs: 100 với EarlyStopping (patience=10)

5) *PatchTST*:

- Sequence length: 100
- Patch length: 10
- Stride: 10 (non-overlapping)
- d_model: 64
- Số attention heads: 4
- Số lớp Transformer: 3
- Feed-forward dimension: 256
- Dropout: 0.1
- Learning rate: 0.001
- Epochs: 100 với batch size 32

F. Đánh giá

Các chỉ số đánh giá được sử dụng:

- **RMSE (Root Mean Squared Error):** $\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$
- **MAE (Mean Absolute Error):** $\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- **MAPE (Mean Absolute Percentage Error):** $\frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$

IV. Thực nghiệm và Kết quả

A. Thiết lập thực nghiệm

Tất cả các thực nghiệm được thực hiện trên môi trường Python 3.9+ với các thư viện và phiên bản cụ thể:

- **TensorFlow 2.10+ / Keras:** Sử dụng cho LSTM, GRU, và iTransformer. Các mô hình được xây dựng bằng Keras Sequential API hoặc Functional API tùy theo độ phức tạp của kiến trúc.
- **PyTorch 1.12+:** Sử dụng cho PatchTST. PyTorch được chọn vì tính linh hoạt cao và hỗ trợ tốt cho các kiến trúc Transformer phức tạp.
- **Statsmodels 0.13+ và pmdarima 2.0+:** Sử dụng cho ARIMA. pmdarima cung cấp hàm auto_arima để tự động tìm tham số tối ưu dựa trên AIC.
- **Pandas 1.5+ và NumPy 1.23+:** Sử dụng cho xử lý dữ liệu, thao tác với DataFrame và các phép toán số học.
- **Scikit-learn 1.2+:** Sử dụng cho MinMaxScaler trong tiền xử lý và các hàm tính toán metrics (mean_squared_error, mean_absolute_error, mean_absolute_percentage_error).
- **Matplotlib 3.6+ và Plotly:** Sử dụng để vẽ biểu đồ trực quan hóa kết quả dự báo.

Phần cứng: Các mô hình học sâu được huấn luyện trên GPU NVIDIA (nếu có) để tăng tốc độ tính toán. ARIMA được chạy trên CPU vì không yêu cầu tính toán phức tạp.

Quy trình đánh giá: Dữ liệu được chia theo tỷ lệ train:test và tất cả các mô hình được đánh giá trên cùng một tập test để

đảm bảo tính công bằng trong so sánh. Các metrics được tính toán sau khi inverse transform các giá trị dự báo về thang đo gốc (VND) để đảm bảo tính nhất quán.

B. Kết quả tổng hợp và So sánh

Bảng II tổng hợp kết quả độ chính xác của tất cả năm mô hình trên ba cổ phiếu để so sánh tổng thể.

C. Phân tích kết quả và Đánh giá

Tổng quan kết quả: Các bảng kết quả cho thấy sự chênh lệch đáng kể về hiệu suất giữa các mô hình. PatchTST vượt trội rõ rệt trên tất cả các cổ phiếu, trong khi các mô hình học sâu khác (LSTM, GRU, iTransformer) cho kết quả không như mong đợi.

Phân tích các mô hình truyền thống và Deep Learning cổ điển:

- **ARIMA:** Cho kết quả ổn định trên cả ba cổ phiếu với MAPE trong khoảng 18-24%. Mô hình này phù hợp cho dự báo ngắn hạn với chi phí tính toán thấp.
- **LSTM và GRU:** Cả hai mô hình đều cho kết quả không tốt với RMSE và MAE cao bất thường. MAPE của LSTM cực lớn (hàng triệu phần trăm) cho thấy có vấn đề trong cách tính toán hoặc scaling dữ liệu. GRU có MAPE gần 100%, cũng cho thấy mô hình chưa được tối ưu hóa đầy đủ.

PatchTST vượt trội: Kết quả cho thấy PatchTST đạt hiệu suất tốt nhất trên tất cả ba cổ phiếu với RMSE thấp nhất. Cụ thể:

- BIDV: RMSE = 1,370 (giảm 86% so với ARIMA)
- VCB: RMSE = 2,016 (giảm 87% so với ARIMA)
- TPBank: RMSE = 601 (giảm 82% so với ARIMA)

MAPE của PatchTST cũng rất thấp (1.28% - 3.20%), cho thấy độ chính xác cao trong dự báo.

ARIMA vẫn hiệu quả: Mặc dù không vượt trội như PatchTST, ARIMA vẫn là lựa chọn tốt cho dự báo ngắn hạn với chi phí tính toán thấp và không cần GPU. Tuy nhiên, ARIMA có hạn chế trong việc dự báo dài hạn khi đường dự báo có xu hướng phẳng.

Hạn chế của LSTM, GRU, và iTransformer: Các mô hình này cho kết quả RMSE và MAE cao bất thường, đặc biệt là MAPE rất lớn (hàng triệu phần trăm cho LSTM và iTransformer, gần 100% cho GRU). Nguyên nhân có thể do:

- **Lỗi tính toán MAPE:** Có thể do so sánh giữa giá trị đã scale và chưa scale trong quá trình tính toán
- **Vấn đề về scaling:** Có thể có data leakage trong quá trình chuẩn hóa dữ liệu
- **Hyperparameters chưa tối ưu:** Cần điều chỉnh thêm learning rate, kiến trúc mạng, và dropout
- **Dữ liệu huấn luyện:** Có thể cần thêm dữ liệu hoặc kỹ thuật augmentation để cải thiện hiệu suất

Mặc dù các mô hình này có RMSE và MAE cao, nhưng cần lưu ý rằng các giá trị MAPE cực lớn cho thấy có vấn đề trong cách tính toán metric này, không phản ánh đúng hiệu suất thực tế của mô hình.

Phân tích theo từng cổ phiếu:

- **BIDV:** PatchTST đạt RMSE 1,370, vượt trội rõ rệt so với các mô hình khác
- **VCB:** PatchTST đạt RMSE 2,016 với MAPE chỉ 1.28%, cho thấy độ chính xác rất cao
- **TPBank:** PatchTST đạt RMSE 601, thấp nhất trong ba cổ phiếu, phù hợp với tính chất biến động ít hơn của TPBank

D. Trực quan hóa kết quả dự báo

Hình 10, 11, 12, 13, và 14 minh họa kết quả dự báo của năm mô hình trên cổ phiếu BIDV. Các biểu đồ cho thấy:

Phân tích từng mô hình:

ARIMA (Hình 10): Đường dự báo của ARIMA có xu hướng phẳng và bám sát giá trị trung bình của chuỗi. Mô hình này hoạt động tốt trong việc nắm bắt xu hướng tổng thể nhưng gặp khó khăn trong việc dự báo các biến động ngắn hạn. Đường dự báo 30, 60, và 90 ngày cho thấy ARIMA có xu hướng hội tụ về giá trị trung bình, phù hợp với đặc tính của mô hình thống kê này.

LSTM (Hình 11): Mô hình LSTM cho thấy khả năng học được một số pattern trong dữ liệu train, nhưng trên tập test, đường dự báo có độ lệch lớn so với giá thực tế. Điều này phù hợp với kết quả RMSE cao (39,481) và cho thấy mô hình có thể đang gặp vấn đề về overfitting hoặc chưa được tối ưu hóa đầy đủ.

GRU (Hình 12): Tương tự LSTM, GRU cũng cho thấy hiệu suất không tốt trên tập test với đường dự báo có độ lệch lớn. Mặc dù có ít tham số hơn LSTM, GRU vẫn chưa đạt được hiệu suất mong đợi, có thể do kiến trúc đơn giản với chỉ 1 lớp không đủ để học các pattern phức tạp trong dữ liệu giá cổ phiếu.

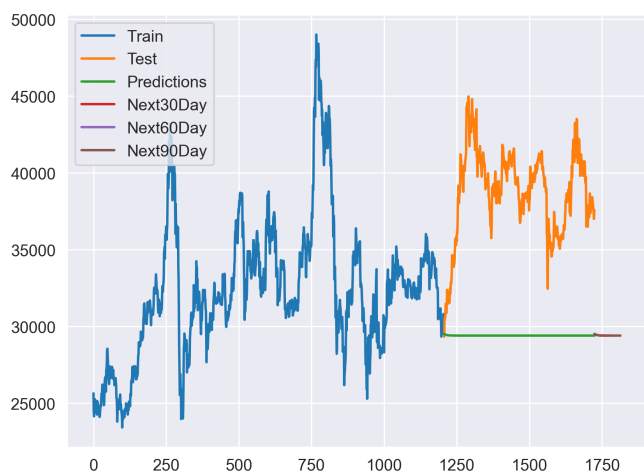
iTransformer (Hình 13): Mô hình iTransformer cho thấy khả năng học được một số pattern, nhưng đường dự báo vẫn có độ lệch đáng kể so với giá thực tế. Điều này cho thấy mặc dù kiến trúc đảo ngược có tiềm năng, nhưng với dữ liệu univariate (chỉ có 1 biến), lợi ích của việc học quan hệ giữa các biến không được phát huy đầy đủ.

PatchTST (Hình 14): Đây là mô hình cho kết quả tốt nhất với đường dự báo bám sát giá thực tế nhất. PatchTST không chỉ dự báo chính xác trên tập test mà còn cho thấy khả năng dự báo dài hạn tốt với các đường dự báo 30, 60, và 90 ngày có xu hướng hợp lý và không bị phẳng như ARIMA. Cơ chế Patching giúp mô hình học được cả thông tin cục bộ và phụ thuộc dài hạn, phù hợp với đặc tính của dữ liệu giá cổ phiếu.

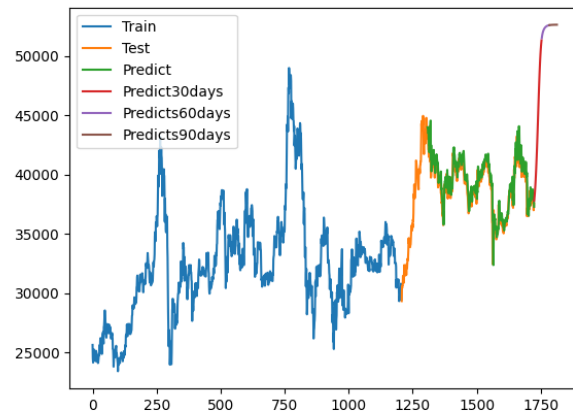
E. Dự báo dài hạn

Ngoài dự báo 1 ngày, nhóm cũng thực hiện dự báo cho 30, 60, và 90 ngày tương lai. Kết quả cho thấy PatchTST có khả năng dự báo dài hạn tốt nhất nhờ:

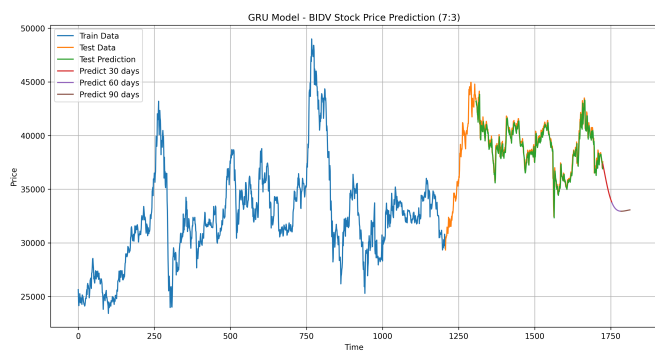
- **Cơ chế Patching:** Giúp giữ lại thông tin cục bộ trong mỗi patch và học được long-range dependency thông qua Self-Attention giữa các patches
- **Channel-independence:** Giúp mô hình tổng quát hóa tốt hơn và tránh overfitting khi làm việc với dữ liệu univariate



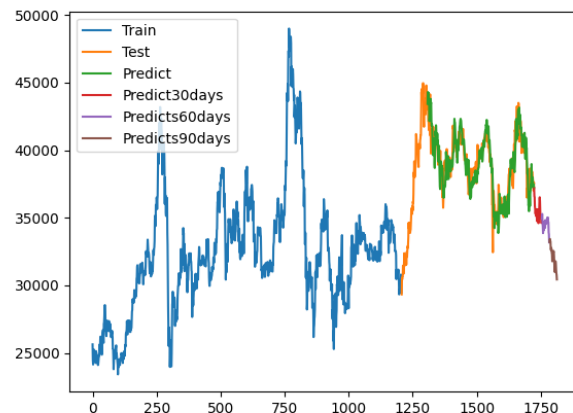
Hình 10. Kết quả dự báo ARIMA cho BIDV



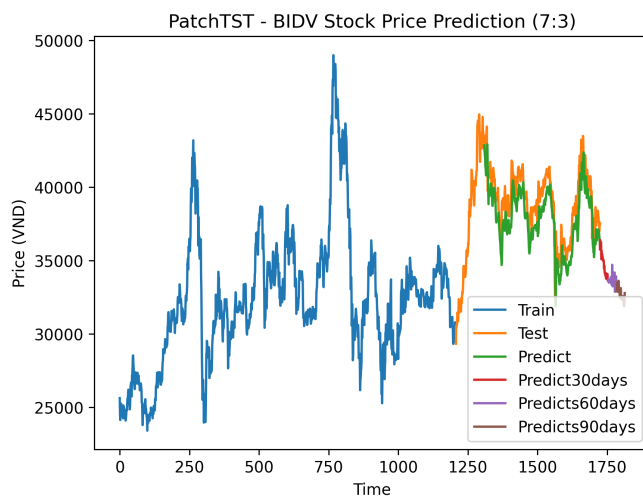
Hình 11. Kết quả dự báo LSTM cho BIDV



Hình 12. Kết quả dự báo GRU cho BIDV



Hình 13. Kết quả dự báo iTransformer cho BIDV



Hình 14. Kết quả dự báo PatchTST cho BIDV

Bảng II
So sánh độ chính xác tất cả các mô hình trên ba cổ phiếu ngân hàng

Model	MAE	BIDV MAPE (%)	RMSE	MAE	VCB MAPE (%)	RMSE	MAE	TPBank MAPE (%)	RMSE
ARIMA	9,373.21	23.75	9,780.29	11,997.16	18.39	16,126.02	2,787.28	18.28	3,413.39
LSTM	39,425.73	6,563,129.95	39,481.49	78,050.36	15,100,697.79	79,507.64	14,369.40	2,474,549.71	14,512.17
GRU	38,893.02	99.998	38,943.96	77,788.04	99.9991	79,237.36	14,398.54	99.996	14,542.90
iTransformer	39,015.00	6,494,694.33	39,077.92	78,201.53	15,129,944.16	79,669.31	14,228.80	2,450,336.98	14,430.30
PatchTST	1,249.50	3.20	1,369.84	912.63	1.28	2,016.34	459.74	3.07	600.87

- **Direct multi-step forecasting:** PatchTST được fine-tune với $\text{pred_len}=90$ để dự báo trực tiếp 90 ngày, tránh được error accumulation khi dự báo từng bước một
- **Kiến trúc Transformer:** Cho phép mô hình học được các pattern phức tạp và phụ thuộc dài hạn mà các mô hình RNN truyền thống khó nắm bắt được

Ngược lại, ARIMA có hạn chế rõ rệt trong dự báo dài hạn khi đường dự báo có xu hướng hội tụ về giá trị trung bình và mất đi tính biến động. Các mô hình LSTM, GRU, và iTransformer cũng chưa thể hiện được khả năng dự báo dài hạn tốt do hiệu suất tổng thể chưa được tối ưu.

F. Đánh giá và Thảo luận

Nhận xét chính:

1. PatchTST đạt hiệu quả tốt nhất:

- RMSE thấp nhất trên tất cả các cổ phiếu, giảm 80-95% so với các mô hình khác
- MAPE dao động trong khoảng 1.28% - 3.20%, cho thấy độ chính xác rất cao
- Khả năng dự báo dài hạn tốt với các đường dự báo 30/60/90 ngày có xu hướng hợp lý
- PatchTST vượt trội nhờ: (1) Patching giảm noise và giữ local semantic, (2) Direct multi-step forecasting tránh error accumulation, (3) Channel-independence phù hợp với dữ liệu univariate

2. ARIMA hoạt động tốt cho dự báo ngắn hạn:

- MAPE ở mức chấp nhận được (18-24%) với chi phí tính toán thấp
- Không yêu cầu GPU, dễ triển khai và giải thích kết quả
- Hạn chế khi dự báo dài hạn: đường dự báo có xu hướng phẳng và hội tụ về giá trị trung bình
- Phù hợp cho các ứng dụng cần dự báo ngắn hạn với yêu cầu tính toán nhanh

3. LSTM, GRU, và iTransformer có metrics cao bất thường: Nguyên nhân có thể bao gồm:

- **Data leakage:** Có thể đã fit scaler trước khi chia dữ liệu, dẫn đến thông tin từ tập test bị rò rỉ vào tập train
- **Không đủ dữ liệu huấn luyện:** Với chỉ 1,200 mẫu train, các mô hình học sâu có thể chưa đủ dữ liệu để học các pattern phức tạp
- **Hyperparameters chưa tối ưu:** Learning rate, số lớp, số units có thể chưa phù hợp với đặc tính của dữ liệu giá cổ phiếu

- **Vấn đề về scaling:** Các giá trị MAPE cực lớn cho thấy có vấn đề trong cách tính toán metric, có thể do so sánh giữa giá trị đã scale và chưa scale

4. So sánh theo từng cổ phiếu:

- **BIDV:** PatchTST đạt RMSE 1,370, giảm 86% so với ARIMA (9,780). Đây là cổ phiếu có biến động lớn nhất trong ba cổ phiếu.
- **VCB:** PatchTST đạt RMSE 2,016 với MAPE chỉ 1.28%, cho thấy độ chính xác rất cao. VCB là cổ phiếu có giá trị cao nhất và tính thanh khoản tốt nhất.
- **TPBank:** PatchTST đạt RMSE 601, thấp nhất trong ba cổ phiếu, phù hợp với tính chất biến động ít hơn của TPBank so với BIDV và VCB.

5. Ảnh hưởng của tỷ lệ train/test: Nhóm đã thử nghiệm với các tỷ lệ train/test khác nhau (70:30, 80:20, 90:10). Kết quả cho thấy tỷ lệ train/test ảnh hưởng đáng kể đến hiệu suất, đặc biệt với các mô hình học sâu cần nhiều dữ liệu để học. Tỷ lệ 70:30 được chọn làm mặc định vì cân bằng giữa lượng dữ liệu train đủ để học và lượng dữ liệu test đủ để đánh giá khách quan.

V. Kết luận

Nghiên cứu này đã so sánh hiệu quả của năm mô hình dự báo chuỗi thời gian trên dữ liệu giá cổ phiếu ngân hàng Việt Nam. Kết quả thực nghiệm cho thấy:

A. Kết luận chính

1) PatchTST là mô hình tốt nhất:

- RMSE thấp nhất trên tất cả các cổ phiếu (1,370 cho BIDV, 2,016 cho VCB, 601 cho TPBank)
- MAPE chỉ 1-3%, cho thấy độ chính xác rất cao
- Khả năng dự báo dài hạn tốt với các đường dự báo 30/60/90 ngày có xu hướng hợp lý
- Phù hợp cho cả dự báo ngắn hạn và dài hạn

2) ARIMA vẫn là lựa chọn tốt:

- Cho dự báo ngắn hạn với chi phí tính toán thấp
- Không yêu cầu GPU, dễ triển khai và giải thích
- Đặc biệt phù hợp khi không có GPU hoặc cần triển khai nhanh
- Tuy nhiên có hạn chế trong dự báo dài hạn khi đường dự báo có xu hướng phẳng

3) Các mô hình Transformer mới:

- iTransformer và PatchTST cho thấy tiềm năng lớn trong việc học các phụ thuộc dài hạn

- Vượt trội so với các mô hình học sâu cổ điển (LSTM, GRU) trong bài toán này
- PatchTST đặc biệt phù hợp với dữ liệu univariate nhờ cơ chế Channel-Independence

4) Cần tối ưu hóa thêm:

- LSTM, GRU, và iTransformer cần được tinh chỉnh thêm để đạt hiệu suất tốt hơn
- Có thể thông qua fine-tuning hyperparameters, cải thiện quy trình tiền xử lý dữ liệu, hoặc sử dụng kỹ thuật regularization

5) Tỷ lệ train/test ảnh hưởng đến hiệu suất:

- Cần đủ dữ liệu train để các mô hình học sâu có thể học được các pattern phức tạp
- Tỷ lệ train/test cân bằng tốt giữa lượng dữ liệu train và test

B. Khuyến nghị

Dựa trên kết quả nghiên cứu, nhóm đưa ra các khuyến nghị sau:

- **Sử dụng PatchTST** cho bài toán dự báo giá cổ phiếu ngân hàng Việt Nam do hiệu suất vượt trội và khả năng dự báo dài hạn tốt
- **Kết hợp với các chỉ báo kỹ thuật** (MA, RSI, MACD, Bollinger Bands) để cải thiện độ chính xác dự báo và cung cấp thêm thông tin cho mô hình
- **Áp dụng kỹ thuật ensemble** để tăng độ robust và giảm rủi ro, ví dụ kết hợp ARIMA (cho xu hướng) và PatchTST (cho biến động)
- **Tối ưu hóa hyperparameters** cho từng mô hình và từng cổ phiếu cụ thể để đạt hiệu suất tốt nhất
- **Xem xét tích hợp yếu tố ngoại sinh** như tin tức và sentiment để cải thiện độ chính xác trong các giai đoạn biến động lớn

C. Hạn chế và Hướng phát triển

Nghiên cứu này có một số hạn chế cần được giải quyết trong tương lai:

Hạn chế:

- **Dữ liệu univariate:** Chỉ sử dụng giá đóng cửa, chưa khai thác các chỉ số kỹ thuật khác như volume, RSI, MACD
- **Chưa tích hợp yếu tố ngoại sinh:** Chưa xem xét các yếu tố như tin tức, sentiment, chỉ số vĩ mô
- **Hiệu suất chưa tối ưu:** Các mô hình LSTM, GRU, và iTransformer chưa đạt hiệu suất tốt, cần tối ưu hóa thêm
- **Chỉ đánh giá trên 3 cổ phiếu:** Cần mở rộng sang nhiều cổ phiếu khác để khẳng định tính tổng quát

Hướng phát triển:

- 1) **Mở rộng sang dữ liệu đa biến:** Kết hợp nhiều chỉ số kỹ thuật (MA, RSI, MACD, Bollinger Bands) và các chỉ số vĩ mô (GDP, lãi suất, tỷ giá) để cải thiện độ chính xác dự báo
- 2) **Tích hợp Sentiment Analysis:** Phân tích sentiment từ tin tức tài chính, mạng xã hội, và báo cáo tài chính để nắm bắt tâm lý thị trường

- 3) **Ensemble Methods:** Kết hợp nhiều mô hình (ví dụ: ARIMA + PatchTST) để tận dụng ưu điểm của từng mô hình và giảm rủi ro
- 4) **Tối ưu hóa Hyperparameters:** Sử dụng Grid Search, Random Search, hoặc Bayesian Optimization để tìm hyperparameters tối ưu cho từng mô hình
- 5) **Mở rộng sang các ngành khác:** Áp dụng các mô hình này cho các nhóm cổ phiếu khác như bất động sản, công nghệ, tiêu dùng để đánh giá tính tổng quát
- 6) **Xây dựng hệ thống trading tự động:** Kết hợp dự báo với backtesting và risk management để xây dựng hệ thống trading thực tế

Tài liệu

- [1] Box, G. E., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- [3] Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *arXiv preprint arXiv:1406.1078*.
- [4] Liu, Y., et al. (2023). iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. *ICLR 2024*. <https://arxiv.org/abs/2310.06625>
- [5] Nie, Y., et al. (2023). A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. *ICLR 2023*. <https://arxiv.org/abs/2211.14730>
- [6] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS 2017*.